Trabajo Previo Laboratorio 07:
Introducción a Python y Google Colab

Felipe Colli Olea

4 de mayo de 2026

1. Resumen de la ejecución de los ejemplos

Al ejecutar el cuaderno (*notebook*) de ejemplo proporcionado en Google Colab, se obtuvieron resultados orientados a la manipulación algebraica y visualización de datos utilizando el lenguaje Python:

- **Generación de Vectores:** Mediante la librería `numpy`, se crearon dos arreglos unidimensionales (vectores x e y) compuestos por 10 números reales pseudoaleatorios.
- **Cálculo Numérico:** Se ejecutaron exitosamente operaciones aritméticas básicas (suma, resta, multiplicación, división y potencias). El intérprete devolvió en tiempo real los arreglos resultantes de cada bloque de código.
- **Visualización Gráfica:** Empleando la librería `matplotlib.pyplot`, se renderizaron dos figuras: un gráfico de dispersión (*scatter plot*) relacionando los vectores aleatorios x e y, y un gráfico de líneas continuas representando una función seno en el intervalo de 0 a 2π.

2. Mecánica de Operaciones con Vectores en Python

El comportamiento de Python al operar con estructuras de datos matemáticas (gracias a `numpy`) difiere drásticamente de las listas iterables tradicionales, implementando un mecanismo altamente optimizado a nivel de C:

Operaciones entre un Vector y un Escalar (*Broadcasting*)

Cuando se realiza una operación matemática (como $x + 1$ o $x \cdot 2$) entre un vector y un escalar, Python utiliza un concepto estructural conocido como **Broadcasting**. El escalar se "transmite"virtualmente en memoria para igualar la longitud del vector, aplicando la operación aritmética individualmente a **cada uno de los elementos**, devolviendo un nuevo vector de la misma dimensión sin necesidad de escribir bucles `for` explícitos.

Operaciones entre Vectores (*Element-wise*)

Al operar dos vectores del mismo tamaño (como $x + y$ o x/y), `numpy` no asume operaciones de álgebra lineal pura por defecto (como el producto punto o cruz), sino operaciones **elemento a elemento** (*element-wise*). Los elementos de índices homólogos se operan entre sí de forma estrictamente paralela.

Felipe Colli, Todos los Derechos Reservados

3. Comentarios personales sobre Google Colab y Python

A nivel personal, las instrucciones y lógicas matemáticas presentadas en este entorno introductorio resultan conceptualmente elementales dado mi historial formativo, aunque supusieron un agradable reencuentro con la sintaxis del lenguaje.

Mi inicio en la programación fue precisamente con Python durante la pandemia (2020), desarrollando *scripts* lógicos básicos. Posteriormente, hacia el año 2022, escalé hacia proyectos de Visión Computacional, donde logré implementar la lógica de un vehículo autónomo en el entorno de simulación *Duckietown* utilizando `OpenCV` y `numpy`. Si bien la sintaxis exacta de dichas librerías se ha ido de mi memoria tras un periodo sin utilizar el lenguaje, la intuición matemática persiste: comprendo que una imagen no es más que una matriz multidimensional (un tensor) de píxeles, la cual debe ser transformada de un espacio de color RGB a HSV para filtrar rangos numéricos con precisión, y aplicar operaciones matriciales como desenfoques Gaussianos para calcular trayectorias. Entender esa base matemática hace que el *broadcasting* de vectores 1D visto en este laboratorio sea completamente intuitivo.

El olvido de la sintaxis de Python se debe a que retomé la programación de lleno recién en 2024, con un enfoque diametralmente distinto: la **Programación Competitiva** utilizando **C++**. Durante estos últimos dos años, he participado en la *Olimpiada Chilena de Informática (OCI)*, obteniendo doble medalla de bronce y formando parte de los campamentos clasificatorios para la *IOI (International Olympiad in Informatics)*. En este ecosistema, la exigencia de optimización asintótica estricta y el control manual de la memoria mediante la *Standard Template Library (STL)* generan un contraste evidente con la abstracción de Python.

Respecto a **Google Colab** y los IDEs en la nube, mi postura es pragmática. Herramientas web que en su momento utilicé (como *Replit*) han sufrido una severa degradación en su experiencia de usuario debido a la sobrecarga de integraciones de Inteligencia Artificial. Por lo mismo, mi flujo de trabajo ideal se mantiene estrictamente en un entorno **local**, ejecutando código de forma nativa desde la terminal en Arch Linux utilizando `Neovim`. A pesar de ello, concedo que Colab cumple un propósito académico fenomenal al eliminar la fricción de instalar librerías matemáticas pesadas en equipos institucionales.